Section A: Flask Extensions for Database Access (10 Questions)

1. What is the role of Flask-SQLAlchemy, and how does it simplify database interactions?

   Flask-SQLAlchemy is an extension that integrates SQLAlchemy ORM with Flask. It simplifies database tasks like defining models, relationships, and queries using Python classes instead of raw SQL.

2. Write a Flask model class Student with fields id, name, and age using SQLAlchemy ORM.

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50))
    age = db.Column(db.Integer)
```

3. Explain the difference between db.session.add() and db.session.commit().

   - db.session.add() just stages the object for insertion.
   - db.session.commit() actually saves it permanently to the database.

4. How can you handle database migrations in Flask? Which extension is commonly used?

   We use Flask-Migrate, which is built on Alembic. It tracks changes to models and applies them to the database with commands like flask db migrate and flask db upgrade.

5. Write a query to fetch all records from a User model where age > 18.

```
users = User.query.filter(User.age > 18).all()
```

6. What error occurs if you forget to commit after adding a record in SQLAlchemy?

   If you forget commit(), the changes stay only in memory and won't be written to the database. The main error is data not being saved, sometimes leading to "PendingRollbackError" if rollback is triggered.

7. Demonstrate how to configure a SQLite database URI in Flask.

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///students.db'
```

8. Explain the difference between Flask-SQLAlchemy and Flask-PyMongo.

   - Flask-SQLAlchemy → for relational databases (PostgreSQL, MySQL, SQLite).
   - Flask-PyMongo → for MongoDB (NoSQL database).

9. How do you establish a one-to-many relationship (e.g., User has many Posts) in SQLAlchemy?

```
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    posts = db.relationship('Post', backref='author', lazy=True)
class Post(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'))
```

10. Debug: A Flask app crashes saying No application found. What might be the issue when initializing SQLAlchemy?

    This happens if SQLAlchemy is initialized before binding the app. You must call db.init_app(app) inside the app context.

Section B: Django REST Framework (DRF) Basics (10 Questions)

11. What is a Serializer in DRF, and why is it needed?

    A serializer converts complex data types like Django models into JSON for APIs, and also validates/deserialize incoming JSON to Python objects.

12. Write a serializer for a Book model having id, title, and author.

```
from rest_framework import serializers
class BookSerializer(serializers.Serializer):
    id = serializers.IntegerField()
    title = serializers.CharField(max_length=100)
    author = serializers.CharField(max_length=50)
```

13. Difference between APIView and ViewSet in DRF.

    - APIView → gives full control, write methods like get() and post().
    - ViewSet → automatically provides CRUD operations with less code, usually paired with routers.

14. Explain how ModelSerializer reduces boilerplate code.

    ModelSerializer reduces boilerplate by automatically generating fields from the model definition, so you don't need to write them manually.

15. Write an API endpoint to list all students using DRF's ListAPIView.

```
from rest_framework.generics import ListAPIView
from .models import Student
from .serializers import StudentSerializer
class StudentListView(ListAPIView):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
```

16. How do you add authentication to a DRF API using permissions.IsAuthenticated?

```
from rest_framework.permissions import IsAuthenticated
class StudentListView(ListAPIView):
    permission_classes = [IsAuthenticated]
```

17. What is the role of @api_view decorator in DRF?

    It's used for function-based views in DRF. It tells DRF which HTTP methods (GET, POST, etc.) a function accepts.

18. When would you use GenericAPIView vs APIView?

    - APIView → low-level, more control.
    - GenericAPIView → adds helpful features like queryset, serializer_class, pagination. Used with mixins for CRUD.

19. Debug: Your DRF API returns 403 Forbidden. What are possible reasons?

    - Missing authentication token.
    - User not logged in.
    - User lacks proper permissions.

20. How do you test DRF APIs using Django's test framework?

```
from rest_framework.test import APIClient
client = APIClient()
response = client.get('/students/')
```

Section C: Flask-RESTful Basics (10 Questions)

21. What is Resource in Flask-RESTful?

A Resource represents an API endpoint (like /students). Each resource maps HTTP methods (GET, POST, PUT, DELETE) to class methods.

22. Write a simple Flask-RESTful API with GET method returning {"msg": "Hello World"}.

```
from flask_restful import Resource
class Hello(Resource):
    def get(self):
        return {"msg": "Hello World"}
```

23. Explain how reqparse is used in Flask-RESTful.

reqparse is used to parse and validate request arguments, like form data or JSON inputs.

24. How do you handle 404 Not Found in Flask-RESTful?

Use abort(404, message="Item not found").

25. Write a POST API in Flask-RESTful to add a new book (fields: title, author).

```
class Book(Resource):
    def post(self):
        data = request.get_json()
        return {"title": data['title'], "author": data['author']}
```

26. Difference between Flask's route-based API creation and Flask-RESTful approach.

- Flask routes use @app.route() decorators.
- Flask-RESTful uses **resources + API object** for cleaner and scalable APIs.

27. Debug: A Flask-RESTful POST API returns 415 Unsupported Media Type. What's wrong?

This usually happens if you forget to send Content-Type: application/json in request headers.

28. How can you enable JSON responses globally in Flask-RESTful?

Set app.config['RESTFUL_JSON'] = {'ensure_ascii': False} so responses are always JSON.

29. Explain how to use marshal_with and fields in Flask-RESTful.

```
from flask_restful import fields, marshal_with
resource_fields = {
  'id': fields.Integer,
  'name': fields.String
}
@marshal_with(resource_fields)
def get(self):
  return student_object
```

30. What's the purpose of abort() in Flask-RESTful?

abort() immediately stops request processing and returns an error code (like 400, 404, 500) with a message.

Section D: Advanced API Design (10 Questions)

31. Compare Django REST Framework vs Flask-RESTful. Which suits large-scale projects better and why?

- DRF → better for **large-scale projects**, comes with authentication, permissions, serializers, browsable API.
- Flask-RESTful → lightweight, good for small/medium projects.

32. What is pagination, and how do you implement it in DRF?

```
REST_FRAMEWORK = {
  'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
  'PAGE_SIZE': 10
}
```

33. Implement token-based authentication in DRF.

Install rest_framework.authtoken, add to INSTALLED_APPS, run migrations, and use tokens for API requests.

34. Write a Flask-RESTful API that accepts query parameters (?name=John&age=30).

```
class Hello(Resource):
  def get(self):
    name = request.args.get('name')
    age = request.args.get('age')
    return {"name": name, "age": age}
```

35. How do you version APIs in Django REST Framework?

```
REST_FRAMEWORK = {
  'DEFAULT_VERSIONING_CLASS': 'rest_framework.versioning.NamespaceVersioning',
}
```

36. What are throttling policies in DRF, and why are they important?

They limit request rates to prevent abuse. Example: UserRateThrottle → max requests per user per minute.

37. Write a DRF ViewSet with routes for CRUD operations on a Student model.

```
from rest_framework.viewsets import ModelViewSet
from .models import Student
from .serializers import StudentSerializer
```

```
class StudentViewSet(ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
```

38. Explain how to integrate Swagger (API documentation) with Django REST Framework.

    Use drf-yasg or drf-spectacular to auto-generate Swagger UI documentation for your APIs.

39. How do you validate incoming data in Flask-RESTful? Give an example.

```
parser = reqparse.RequestParser()
parser.add_argument('age', type=int, required=True, help="Age required")
args = parser.parse_args()
```

40. Design an API system where a **school management app** has Students and Teachers resources. Which framework would you choose (Flask-RESTful vs DRF) and why?

    We choose DRF because it has built-in support for relationships (Students ↔ Teachers), authentication, permissions, and is easier to scale for complex systems.

    models.py

```
from django.db import models
class Teacher(models.Model):
    name = models.CharField(max_length=50)
    subject = models.CharField(max_length=50)
    email = models.EmailField(unique=True)
class Student(models.Model):
    name = models.CharField(max_length=50)
    age = models.IntegerField()
    student_class = models.CharField(max_length=10)
    teacher = models.ForeignKey(Teacher, on_delete=models.CASCADE, related_name="students")
```

Serializers:

```
from rest_framework import serializers
from .models import Student, Teacher
class TeacherSerializer(serializers.ModelSerializer):
    class Meta:
        model = Teacher
        fields = '__all__'
class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = '__all__'
```

ViewSets & Routing:

```
from rest_framework.viewsets import ModelViewSet
from .models import Student, Teacher
from .serializers import StudentSerializer, TeacherSerializer
class StudentViewSet(ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
class TeacherViewSet(ModelViewSet):
    queryset = Teacher.objects.all()
    serializer_class = TeacherSerializer
```

urls.py

```
from rest_framework.routers import DefaultRouter
from .views import StudentViewSet, TeacherViewSet
router = DefaultRouter()
router.register(r'students', StudentViewSet)
router.register(r'teachers', TeacherViewSet)
urlpatterns = router.urls
```